

OrionKV: A Decentralized Distributed Key-Value Store with Quorum-Based Consistency

Sai Tarun Yellamraju
North Carolina State University
Raleigh, North Carolina, USA
vyellam@ncsu.edu

Smeet Nagda
North Carolina State University
Raleigh, North Carolina, USA
snagda@ncsu.edu

Yash Mor
North Carolina State University
Raleigh, North Carolina, USA
ymor2@ncsu.edu

Abstract

Modern distributed applications require storage systems that remain available despite node failures, network partitions, and dynamic cluster membership. Traditional approaches that rely on strong consistency and centralized coordination often sacrifice availability and scalability under these conditions.

This paper presents the design and implementation of a decentralized distributed key-value store that prioritizes availability and fault tolerance. The system employs consistent hashing for data partitioning, replication across multiple nodes, quorum-based read and write operations, and gossip-based membership for decentralized failure detection and cluster management. These mechanisms enable the system to continue serving requests under partial failures while providing tunable trade-offs between consistency and latency.

We implement a prototype and evaluate its behavior under varying quorum configurations, node failures, and dynamic membership changes. Our results demonstrate the impact of quorum choices on latency and availability, the system’s resilience under failures, and the efficiency of data redistribution during node joins. This work provides a practical exploration of availability-first distributed storage design and highlights the trade-offs inherent in decentralized architectures.

1 Introduction

1.1 Motivation

Large-scale distributed systems operate in environments where failures are unavoidable. Nodes may crash, networks may partition, and system membership may change dynamically as clusters scale. Under these conditions, designing storage systems that rely on strict coordination or global agreement can lead to reduced availability and increased latency. The CAP principle formalizes this limitation by stating that a distributed system cannot simultaneously guarantee consistency, availability, and partition tolerance, and must therefore make trade-offs in the presence of network failures [2].

As a result, many distributed storage designs prioritize availability by relaxing strong consistency guarantees. Earlier work on weakly consistent and optimistic replication systems demonstrated that allowing replicas to diverge temporarily, while resolving conflicts asynchronously, enables systems to continue serving requests during failures [13, 15]. These approaches emphasize that maintaining continuous operation is often more critical than enforcing immediate consistency, particularly for large-scale, latency-sensitive applications.

In addition to consistency trade-offs, scalable data placement and system coordination present significant challenges. Consistent

hashing provides a mechanism for distributing data across nodes while minimizing data movement during membership changes, making it well-suited for elastic environments [9]. Similarly, epidemic or gossip-based protocols offer a decentralized approach to membership management and state dissemination, allowing systems to converge without relying on centralized control [6].

Collectively, these ideas motivate the design of decentralized storage systems that prioritize availability, scalability, and fault tolerance. By combining relaxed consistency models with deterministic data placement and decentralized coordination, such systems can maintain high performance and resilience in the presence of failures and dynamic cluster conditions. This project is motivated by the need to study these foundational mechanisms in a simplified setting, enabling a clearer understanding of the trade-offs between consistency, availability, and system complexity.

1.2 Problem Statement

The objective of this project is to design and implement a decentralized distributed key-value store that maintains high availability and scalability under node failures and dynamic cluster membership, while enabling trade-offs between consistency and performance.

Such a system must address several challenges. Data must be partitioned across nodes in a manner that balances load and minimizes movement during membership changes, which can be achieved using consistent hashing [9]. At the same time, data must be replicated across multiple nodes to tolerate failures, introducing trade-offs between latency and consistency when coordinating replicas through quorum-based operations [1].

Furthermore, the system must allow temporary divergence between replicas while ensuring eventual convergence, as explored in weakly consistent and optimistic replication models [13, 15]. Finally, cluster membership must be managed without centralized coordination, requiring scalable mechanisms for state dissemination such as epidemic or gossip-based protocols [6].

This project explores these challenges through a design based on deterministic partitioning, replication, quorum-based coordination, and decentralized membership management, with a focus on availability-first behavior and the resulting trade-offs in system performance and consistency.

2 System Design

2.1 Architecture Overview

OrionKV follows a decentralized peer-to-peer architecture in which every storage node participates in request coordination, replica placement, membership dissemination, and failure handling. The

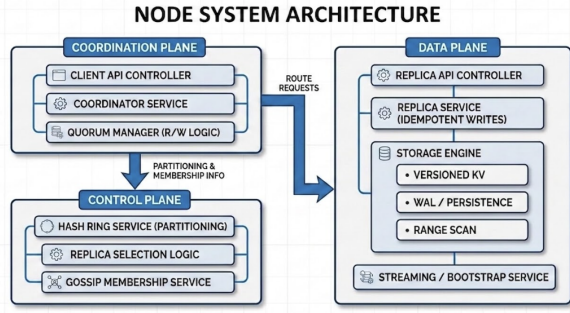


Figure 1: System architecture of OrionKV. The coordination plane handles client-facing request processing and quorum logic, the control plane maintains partitioning and membership state, and the data plane manages replica application, local persistence, range scans, and bootstrap streaming.

system does not rely on a single leader or centralized metadata service. Instead, each node maintains enough local state to route client requests, determine replica ownership, and react to membership changes independently.

At a high level, each node combines three major responsibilities. First, the *coordination plane* receives client requests and computes the replica set for a key using the consistent hash ring. Second, the *data plane* stores replicated records locally, persists them through a write-ahead log, and serves range scans required during rebalancing. Third, the *control plane* manages membership through gossip, detects failed nodes, rebuilds the hash ring, and triggers rebalancing when ownership changes.

The request flow is correspondingly straightforward. A client may contact any available node. That node acts as the coordinator for the operation, hashes the key, determines the responsible replica set, and forwards the request to the selected replicas. For writes, the coordinator waits for enough acknowledgments to satisfy the configured write quorum. For reads, it collects replica responses, reconciles versions using timestamp ordering, and returns the latest value. Since every node executes the same placement and quorum logic, coordination is decentralized and any healthy node can serve as an entry point to the system.

2.2 Consistent Hashing and Data Placement

OrionKV uses consistent hashing to map both nodes and keys into the same token space. The implementation computes tokens using a CRC32-based hash over the UTF-8 representation of the input, producing a deterministic 32-bit value stored as a Java long. Physical nodes are not placed on the ring directly as a single point; instead, each node contributes multiple virtual-node tokens, which are inserted into a globally sorted ring structure.

For a client key k , the coordinator computes its token and finds the first ring token whose value is greater than or equal to the key token. This token identifies the primary placement position for the key. If the key token lies beyond the largest token currently on the ring, placement wraps around to the first token. This circular property ensures that the entire hash space remains covered and that every key is assigned an owner.

The main advantage of this design is that membership changes affect only a subset of keys. When a node joins or leaves, only the keys mapped to the affected token intervals must move; the remainder of the keyspace remains untouched. This sharply reduces redistribution cost compared to naive modulo-based partitioning, where a change in cluster size would invalidate the placement of nearly all keys.

2.3 Virtual Nodes

Virtual nodes are a central mechanism in our implementation. Rather than assigning a physical node a single token on the hash ring, OrionKV assigns each physical node a configurable number of virtual positions. Conceptually, a physical node is the machine or process that stores data, while a virtual node is one ownership point on the ring that maps back to that physical node.

This distinction improves load balance. With only one token per physical node, an unlucky token placement can cause one machine to own a disproportionately large region of the hash space. By assigning many smaller ownership points to each node, the system spreads responsibility more evenly and reduces variance in the amount of data held by each machine. Our experimental evaluation directly reflects this effect: increasing the vnode count reduces the observed imbalance between the most loaded and least loaded nodes.

Virtual nodes also improve rebalancing granularity. When membership changes occur, ownership shifts in smaller token fragments instead of a few large contiguous segments. This helps distribute recovery work across the cluster and avoids concentrating all transferred data on a single donor or receiver. The trade-off is that more virtual nodes create more token boundaries, which increases the number of ranges that the system must reason about during rebalancing.

2.4 Replication and Replica Selection

To tolerate failures, OrionKV replicates each key across multiple distinct physical nodes. The replication factor N determines the number of replicas maintained for a given key. After locating the primary token for a key, the coordinator walks clockwise around the ring and selects the first N distinct node identifiers encountered. Because virtual nodes may map to the same physical machine multiple times, the implementation explicitly removes duplicates during replica selection, ensuring that a single physical node does not count more than once toward the replica set.

This replica selection strategy produces an ordered replica list consisting of the primary owner followed by successive fallback owners. The ordered list is used consistently for writes, reads, and rebalancing. During normal operation, the coordinator forwards a write to all replicas in the replica set and considers the operation successful after receiving the configured number of acknowledgments. For reads, it queries replicas from the same set and reconciles the returned versions.

Replication provides redundancy under both crash and recovery scenarios. If one node fails, remaining replicas can still satisfy quorum reads and writes as long as enough live nodes remain in the replica set. When membership changes remove a replica from the active ring, OrionKV reconstructs the missing replica coverage by

backfilling the newly responsible nodes from surviving replicas. In this way, redundancy is not static; it is continuously re-established as cluster membership evolves.

2.5 Quorum-Based Reads and Writes

OrionKV employs a quorum-based approach to balance consistency, availability, and latency in a replicated key-value store. Each key is replicated across N distinct nodes determined by the consistent hash ring. A write operation is considered successful after acknowledgments are received from at least W replicas, while a read operation returns after collecting responses from at least R replicas.

To ensure consistency, the system enforces the condition:

$$R + W > N$$

This guarantees that any read quorum intersects with any write quorum, ensuring that at least one replica participating in a read contains the most recent successfully written version.

In OrionKV, any node can act as a coordinator. Upon receiving a client request, the coordinator computes the replica set using the hash ring and forwards the request to all N replicas. For write operations, the coordinator waits until W acknowledgments are received before returning success. For read operations, the coordinator collects R responses and performs version reconciliation.

Each stored value is associated with metadata including a timestamp and a deletion marker (tombstone). During read operations, the coordinator compares returned versions and selects the value with the highest timestamp. If the latest version is marked as deleted, the key is treated as absent.

This quorum-based design allows OrionKV to operate under partial failures. Smaller values of R and W reduce latency and improve availability but may result in stale reads if the quorum condition is not satisfied. Larger quorum sizes improve consistency guarantees but increase coordination overhead. Thus, OrionKV exposes quorum configuration as a fundamental trade-off between performance and consistency.

2.6 Data Plane and Versioning

The data plane is responsible for storing, retrieving, and replicating data across nodes in the system. Each node maintains a local key-value store and exposes internal interfaces for handling replica reads, replica writes, and data transfer during rebalancing. To ensure fault tolerance, data is replicated across multiple nodes, allowing the system to continue serving requests despite node failures. Operations are coordinated across replicas using partial quorum mechanisms, enabling the system to reduce latency while providing configurable consistency guarantees [1, 10].

To support scalability under dynamic membership, the data plane incorporates range-based data transfer during node joins and failures. By leveraging consistent hashing, only the affected portions of the keyspace are redistributed, minimizing data movement and reducing system disruption [9, 10]. This allows the system to scale horizontally while maintaining stable performance.

Since replicas may temporarily diverge due to concurrent updates and network delays, the system employs a versioning mechanism to ensure eventual convergence. Each value is associated with

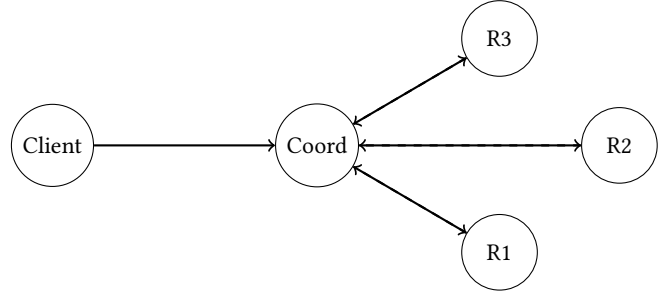


Figure 2: Replica coordination in the data plane. The coordinator forwards requests to replicas, collects responses to satisfy quorum conditions, and resolves versions during reads.

a logical timestamp, enabling the system to determine the relative ordering of updates. Conflict resolution follows a last-write-wins policy, where the value with the highest timestamp is considered authoritative. This approach draws on foundational work in logical time and weakly consistent replication, as well as availability-focused storage system designs [5, 11, 13, 15?].

During read operations, multiple replicas may return different versions of the same key. The coordinator resolves these by selecting the latest version and may propagate this value to stale replicas through asynchronous repair. This design maintains system responsiveness while gradually improving replica consistency, providing a practical balance between availability and correctness in decentralized storage systems [5].

2.7 Gossip Membership and Failure Detection

OrionKV uses a decentralized gossip-based protocol to maintain cluster membership and detect node failures without relying on a centralized coordinator. Each node maintains a local membership table containing the state of other nodes in the system, including their status, last-seen timestamp, and incarnation number.

Nodes can be in one of three states: *ALIVE*, *SUSPECT*, or *DEAD*. Periodically, each node selects a peer and exchanges membership information. Upon receiving gossip messages, nodes merge the incoming state with their local view, preferring newer information based on incarnation numbers and state transitions.

Failure detection is implemented using timeout-based suspicion. If a node has not been observed within a specified interval, it is marked as *SUSPECT*. This intermediate state prevents false positives due to transient network delays. If the node remains unreachable for a longer duration, it is marked as *DEAD*.

To ensure robustness, the gossip mechanism prioritizes communication with nodes marked as *ALIVE*, while still allowing fallback to other reachable peers to support recovery and healing of partially disconnected nodes. This prevents isolated nodes from permanently diverging from the cluster view.

Membership changes trigger reconstruction of the consistent hash ring. Nodes marked as *DEAD* are excluded from replica selection, and newly joined nodes are incorporated into the ring. Since all nodes independently rebuild the ring using the same deterministic logic, the system eventually converges to a consistent routing state once gossip propagation stabilizes.

This design allows OrionKV to achieve scalable and fault-tolerant membership management, where cluster state evolves dynamically and converges without centralized coordination.

2.8 Rebalancing Mechanisms

Rebalancing in OrionKV is the process of redistributing data when cluster membership changes. These changes occur during node joins, failures, and rejoins. The system uses consistent hashing with virtual nodes, ensuring that membership changes affect only a subset of the keyspace rather than requiring a full redistribution of data.

The hash ring partitions the keyspace into token ranges of the form:

$$(prev_token, current_token]$$

Each range is owned by the node associated with the *current_token*. When membership changes, OrionKV compares the previous ring state with the updated ring to determine which token ranges have changed ownership.

Data movement is performed using range-based streaming. Instead of transferring all stored data, only keys that fall within affected token ranges are moved between nodes. Each transferred record includes the key, value, timestamp, and deletion metadata. The receiving node applies incoming records using idempotent versioning logic, ensuring that stale data does not overwrite newer values.

Rebalancing is designed to be localized and incremental. Since virtual nodes divide the keyspace into smaller partitions, data movement is distributed across multiple nodes, reducing hotspots and improving parallelism. This approach minimizes disruption during scaling and failure recovery.

The following sections describe how rebalancing is handled under different cluster events.

2.9 Node Join (Bootstrap)

When a new node joins the cluster, it is first incorporated into the membership system and assigned a set of virtual node tokens. These tokens define the key ranges for which the node becomes responsible. However, since the node initially has no local data, it must acquire the corresponding key ranges from existing nodes.

After the hash ring is rebuilt, OrionKV identifies the ranges that have shifted to the new node. For each such range, the new node initiates bootstrap streaming by requesting data from the previous owners of those ranges. Source nodes scan their local storage and stream matching key-value pairs in batches.

This process allows the joining node to incrementally build its local dataset without requiring a full data copy. Since virtual nodes distribute ownership across multiple parts of the ring, the joining node typically receives data from multiple existing nodes, improving load distribution during bootstrap.

Node join operations are generally more expensive than other rebalancing events because the new node must fetch all assigned data ranges from scratch. The total cost depends on the amount of data transferred, the number of affected token ranges, and network throughput.

2.10 Node Failure

When a node fails, it is first marked as *SUSPECT* and later as *DEAD* by the failure detection mechanism. Once marked *DEAD*, the node is excluded from the hash ring, and its token ranges are reassigned to successor nodes.

Because OrionKV maintains multiple replicas for each key, the system can continue serving requests as long as the required quorum can be achieved. Replica routing automatically shifts to the remaining active nodes after ring reconstruction.

Failure rebalancing focuses on restoring replica completeness rather than transferring all data. Since surviving replicas already hold copies of many keys, only missing replica ranges need to be reconstructed. OrionKV identifies newly required replica ranges and performs range-based backfill from healthy nodes.

This approach reduces recovery cost compared to node join, as data movement is limited to restoring replication guarantees rather than initializing a completely empty node.

2.11 Node Rejoin and Recovery

When a previously failed node rejoins the cluster, it is reintroduced through the membership protocol and participates in ring reconstruction. However, the node may have missed updates while it was unavailable, resulting in incomplete or stale local state.

To address this, OrionKV performs rejoin recovery through range-based backfill. The system determines the token ranges for which the node is responsible and identifies any missing or outdated data. These ranges are then streamed from other replicas in the cluster.

Incoming records are applied using timestamp-based version comparison. If the rejoining node already has a newer version of a key, the incoming record is ignored. Otherwise, the record is applied to update local state.

This mechanism ensures that rejoining nodes eventually converge to the correct replica state without introducing inconsistencies. By leveraging the same streaming and idempotent update logic used during bootstrap, OrionKV maintains correctness while supporting dynamic cluster recovery.

2.12 Implementation Details

The system is implemented in Java 17 using Spring Boot, with each OrionKV node running as an independent service instance. All nodes execute the same application code, while configuration such as node identity, ports, and seed addresses is supplied at runtime, enabling decentralized cluster formation.

The architecture is organized into modular components for request routing, quorum coordination, replication, membership management, gossip, failure detection, rebalancing, and storage. Communication uses a combination of REST/HTTP endpoints for client-facing and administrative operations, and gRPC with Protocol Buffers for efficient inter-node coordination and replica communication.

The storage layer is a persistent key-value store that supports versioned values, tombstones, and range scans. Each record includes a value, timestamp, and deletion metadata, enabling idempotent

updates and last-write-wins conflict resolution. These same mechanisms are used for replica writes, read repair, and streaming-based rebalancing.

Cluster membership and routing are maintained using gossip-based failure detection together with consistent hashing and virtual nodes. The system is built with Maven and deployed using Docker containers, allowing multiple isolated nodes to run across one or more virtual machines for testing and experimentation.

The source code for OrionKV is available in the project repository [17].

3 Experimental Evaluation

3.1 Experimental Setup

Experiments were conducted on infrastructure provisioned through VCL (vcl.ncsu.edu). The deployment consisted of five VCL instances, each hosting five Docker containers running OrionKV nodes, resulting in a total cluster size of twenty-five containerized nodes. All nodes were configured to communicate with one another over the network, allowing the system to operate as a fully connected distributed cluster.

An additional VCL instance was provisioned separately to host the client-router layer. This client-router service maintained connectivity to the OrionKV cluster and was capable of routing requests to any live node in the system. Separating the client layer from the storage cluster allowed client-side request routing and cluster-side node behavior to be evaluated independently.

Each OrionKV node ran the same Java 17 and Spring Boot application inside a Docker container. Runtime configuration parameters specified the node identity, REST port, gRPC port, seed-node information, and the number of virtual nodes used for consistent hashing. In the experimental configuration, the virtual-node count was set to 512 per physical node. Data replication was configured with a replication factor of 3, meaning that each key was stored on three replica nodes. Quorum-based operations used $R = 2$ for reads and $W = 2$ for writes, ensuring that successful read and write quorums intersected while still tolerating a single replica failure for a given key.

The experimental workload consisted of GET and PUT operations issued through the client-router layer, along with scripted smoke tests, bulk-loading utilities, and benchmark scripts. These tools were used to generate application traffic, preload datasets, validate replication behavior, and measure request latency and throughput. Additional scripted scenarios were used to test membership convergence, gossip-based failure detection, quorum routing under failures, node joins, node departures, and range rebalancing.

This setup allowed the system to be evaluated under realistic distributed conditions while remaining reproducible and configurable. The experiments captured both the functional correctness and performance behavior of the system under dynamic cluster conditions.

3.2 Data Distribution Across Nodes

To evaluate how data was replicated across the cluster, we used a custom benchmarking and analysis script executed through the client-router layer. For each request, the script recorded the replica

nodes returned in the response metadata and aggregated how frequently each node appeared as part of a replica set. This provided a practical view of how replicated key-value pairs were distributed across the twenty-five OrionKV nodes during workload execution.

Figure 3 shows the number of replica responses associated with each node. The observed distribution was well balanced across the cluster, with each node participating in a comparable number of replica assignments. This indicates that the consistent hashing and virtual-node based placement strategy distributed replicated data evenly rather than concentrating responsibility on a small subset of machines.

As shown in Figure 3, no node was over or under 20% of the cluster-wide average. This suggests that the replication mechanism achieved good balance in replica placement and avoided significant skew in storage and request participation. Overall, the results demonstrate that the system spread replicated key-value data effectively across all nodes in the cluster.

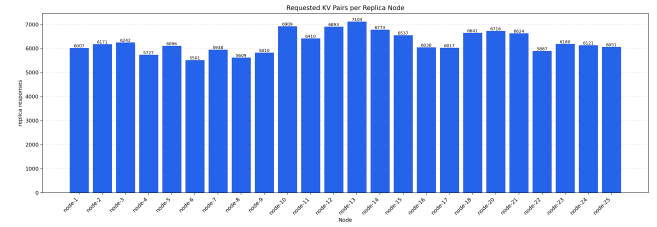


Figure 3: Distribution of replica responses across the 25 OrionKV nodes.

3.3 Rebalancing Cost Analysis

We evaluate rebalancing cost in terms of both *recovery time* and *ownership movement*. Recovery time is measured as the elapsed time reported by the node-local rebalancing metrics endpoint from the start of a membership-triggered transfer until the rebalance completes successfully. Ownership movement is measured using the number of token ranges reassigned during the event. This distinction is important because a rebalance may involve many small ranges or fewer larger ranges; range count therefore captures structural change in the ring, while time captures end-to-end operational cost.

All rebalancing experiments were conducted under a fixed replication factor, fixed quorum configuration, and fixed dataset size. The cluster was first brought to a stable state and preloaded with data. Membership changes were then introduced in a controlled manner by either removing a node from the cluster or reintroducing a previously absent node. For each event, we recorded the reported rebalance duration together with the number of affected ranges. This methodology isolates the effect of topology change from unrelated workload variation.

The measured behavior is asymmetric. Node joins are more expensive than node removals because a joining or rejoining node starts with incomplete local state and must fetch all newly assigned ranges from existing replicas. This requires repeated range scans at donor nodes, batched transfer of records, and replay of incoming updates at the receiver. By contrast, when a node leaves or fails,

surviving replicas already store much of the required data. The system therefore focuses on reconstructing missing replica coverage rather than initializing a cold node from scratch. In operational terms, leave-side recovery is largely a repair process, whereas join-side recovery is a data acquisition process.

This asymmetry should not be interpreted as an implementation artifact alone; it arises directly from the system design. Range-based transfer, replica backfill, and vnode-driven ownership fragmentation all contribute to the observed cost. As a result, rebalancing must be evaluated not only as a correctness mechanism, but also as a performance-sensitive consequence of the chosen partitioning strategy.

3.4 Rebalance vs. VNode Trade-off

We next evaluate how the virtual-node count influences the trade-off between rebalancing cost and distribution quality. The motivation is straightforward: increasing the number of virtual nodes generally improves load balance by giving each physical node more ownership points on the ring, but it also increases the number of token boundaries the system must track and potentially move during membership change.

To study this effect, we varied the vnode count across four configurations: 128, 256, 512, and 1024 virtual nodes per physical node. For each configuration, we kept the cluster size, replication factor, quorum parameters, and total dataset size constant. The system was loaded to a stable state, then subjected to controlled membership events, and the corresponding rebalance metrics were recorded. This ensures that differences across configurations are attributable to vnode granularity rather than to workload or cluster-size drift.

Figure 4 reports the measured rebalance duration for join and leave events as a function of vnode count. The trend is clear: higher vnode counts increase rebalance time, with join operations growing more sharply than leave operations. The reason is that more virtual nodes create more ownership fragments and therefore more range-level bookkeeping and transfer activity during rebalancing. Even when the total logical data volume remains fixed, the system must reason about and process a larger number of ownership intervals.

However, rebalance time alone does not justify the choice of vnode count. Figure 5 complements this result by showing the trade-off between JOIN rebalance time and data imbalance across the cluster. Here, data imbalance is expressed as the maximum-minus-minimum per-node load gap as a percentage of the average node load. Lower values indicate a more even distribution.

The trade-off is therefore explicit. Increasing the vnode count reduces imbalance and improves steady-state fairness, but it also increases the cost of adapting to membership change. In other words, virtual nodes improve distribution quality by making placement more fine-grained, but that same fine granularity raises the operational cost of rebalancing. This result supports the broader claim that vnode configuration is not merely a tuning parameter for fairness; it is also a control knob for recovery overhead.

Taken together, these results show that vnode count should be selected by considering both steady-state balance and dynamic recovery cost. A configuration with too few virtual nodes may leave the cluster unevenly loaded, while a configuration with too many virtual nodes may incur unnecessarily high rebalancing overhead

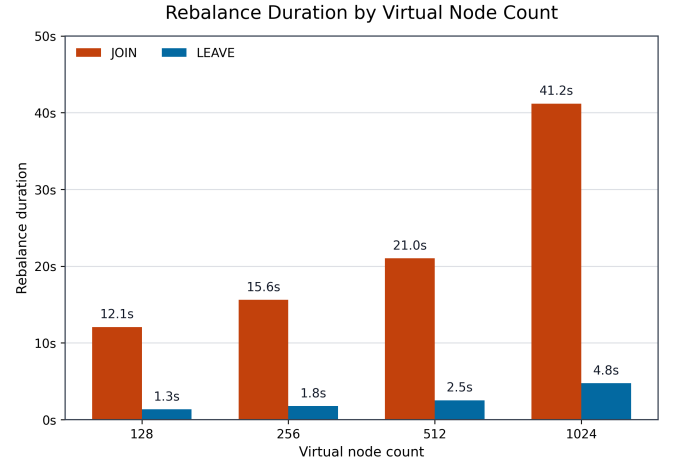


Figure 4: JOIN and LEAVE rebalance duration as the virtual-node count increases. Higher vnode counts increase rebalancing overhead, with JOIN events remaining consistently more expensive than LEAVE events.

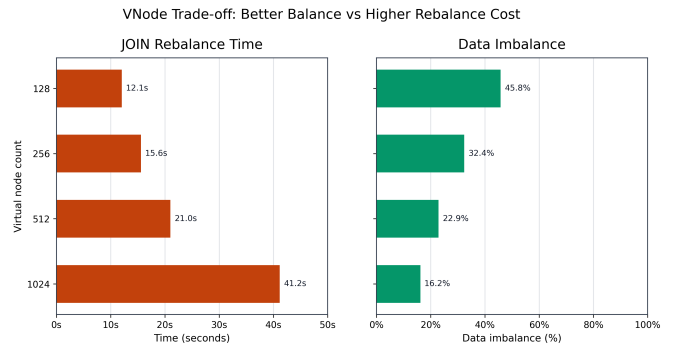


Figure 5: Trade-off between JOIN rebalance time and data imbalance under different vnode configurations. Lower imbalance is achieved at the cost of longer rebalancing time.

during joins, failures, and recoveries. The evaluation therefore motivates vnode count as a design parameter that balances fairness against operational responsiveness.

3.5 Quorum Size vs Latency

We evaluate the impact of quorum size on request latency by measuring both read (GET) and write (PUT) latencies under varying quorum configurations. The replication factor N was fixed, while R and W were varied across different settings.

Figure 6 shows that write latency increases with larger values of W . Since the coordinator must wait for acknowledgments from W replicas before completing the operation, increasing W directly increases coordination overhead. Although requests are issued in parallel, overall latency is bounded by the slowest replica required to satisfy the quorum.

As shown in Figure 7, read latency increases with larger values of R . The coordinator must collect responses from multiple replicas

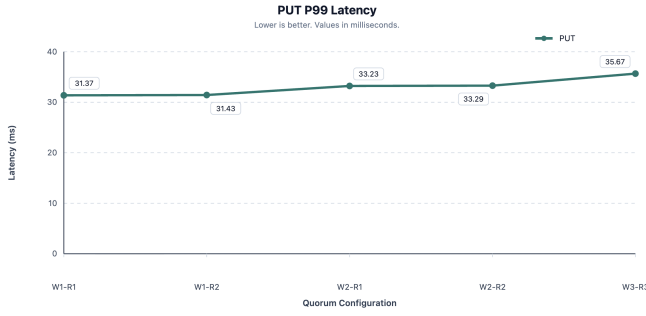


Figure 6: Write (PUT) latency as a function of write quorum size W . Latency increases as the coordinator waits for more replica acknowledgments.

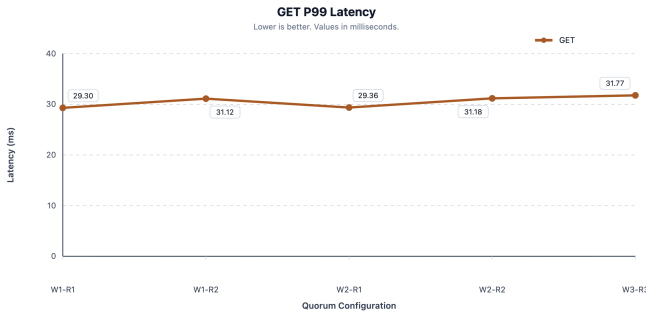


Figure 7: Read (GET) latency as a function of read quorum size R . Latency increases as more replica responses are required for read completion.

and perform version reconciliation before returning a result. Similar to writes, the latency is influenced by the slowest responding replica in the quorum.

Configurations with smaller quorum sizes (e.g., $R = 1$, $W = 1$) achieve the lowest latency for both reads and writes, but do not guarantee that $R + W > N$, and therefore may return stale data. In contrast, configurations satisfying $R + W > N$ ensure overlap between read and write quorums, providing stronger consistency guarantees at the cost of increased latency.

These results demonstrate that read and write latencies scale independently with R and W , and highlight the trade-off between consistency and performance in quorum-based systems.

3.6 Failure Handling and Recovery

We evaluate the behavior of OrionKV under node failures and subsequent recovery to verify system availability and correctness under dynamic membership changes.

During testing, node failures were introduced by terminating one or more nodes in the cluster while the system was actively serving requests. The gossip-based membership protocol first marked failed nodes as *SUSPECT* and eventually transitioned them to *DEAD* after timeout expiration. Once marked *DEAD*, the nodes were excluded

from the consistent hash ring, and replica routing was updated accordingly.

The system continued to serve client requests as long as the configured quorum requirements could be satisfied. For lower quorum configurations, operations remained available even after node failure. For stricter quorum settings, availability depended on whether sufficient replicas remained reachable to form a quorum.

Rebalancing was triggered after membership updates. The system identified newly required replica ranges and performed range-based backfill from surviving nodes. This ensured that replication guarantees were eventually restored after failure.

Node recovery was evaluated by restarting previously failed nodes and allowing them to rejoin the cluster. Upon rejoining, nodes participated in membership propagation and ring reconstruction. Since rejoining nodes may miss updates during downtime, OrionKV performed backfill of missing data using range-based streaming. Incoming records were applied using timestamp-based version comparison to ensure correctness.

Although this evaluation does not provide quantitative latency or throughput measurements under failure, it validates that OrionKV maintains availability under partial failures and restores replica completeness after recovery. These observations confirm the correctness of the system’s failure handling and rebalancing mechanisms.

3.7 Key Observations

Several findings emerged consistently across our experiments. First, consistent hashing with virtual nodes produced a clear improvement in data distribution fairness. Increasing the vnode count reduced skew across the cluster and yielded a more even allocation of records among live nodes. This confirms the intended role of virtual nodes as a load-balancing mechanism rather than merely an implementation detail.

Second, quorum configuration had a direct and predictable impact on latency. Larger write quorums increased PUT latency because the coordinator had to wait for more acknowledgments, while larger read quorums increased GET latency because more replica responses had to be collected and reconciled. These results make the availability–latency–consistency trade-off concrete: stronger quorum overlap improves consistency guarantees, but at the cost of slower operations.

Third, rebalancing behavior was asymmetric across membership events. Node joins and rejoins were consistently more expensive than node removals because the joining node had to reconstruct its assigned state through range-based transfer. In contrast, node failure handling benefited from existing replica redundancy and therefore required less data movement to restore safe replica coverage. Taken together, these observations show that the system’s major design choices—consistent hashing, virtual nodes, replication, and quorum coordination—are tightly coupled and must be evaluated as interacting mechanisms rather than in isolation.

4 Challenges and Lessons Learned

Building OrionKV exposed several practical challenges inherent to decentralized distributed systems. While the core design principles are conceptually straightforward, ensuring correctness under

concurrent operations, failures, and dynamic membership required careful implementation and debugging.

One of the initial challenges was membership instability during startup. Non-seed nodes could attempt to join the cluster before the seed node's RPC services were fully initialized, leading to repeated join failures. This issue was resolved by introducing retry mechanisms and improving startup sequencing to ensure that nodes only attempt to join when the cluster is ready.

Another major challenge was maintaining consistent hash ring views across nodes. Since each node independently constructs the ring based on its local membership table, temporary divergence in gossip state could lead to inconsistent replica selection. This required careful handling of membership convergence and validation of routing correctness once gossip stabilized.

Ensuring quorum correctness also proved non-trivial. It was not sufficient to simply forward requests to replicas; the coordinator had to enforce deterministic replica selection, handle timeouts correctly, and merge responses consistently. In particular, read-path correctness required careful reconciliation of versioned data returned from multiple replicas.

A critical issue was encountered with tombstone handling. Initially, local reads suppressed tombstone values while replica reads returned them, leading to inconsistent behavior during quorum reads. This was resolved by standardizing all read paths to return versioned state and deferring visibility decisions to the coordinator. This ensured consistent interpretation of deleted keys across all read operations.

Rebalancing logic introduced additional complexity. Early implementations focused primarily on primary ownership changes and did not fully account for replica responsibilities. This resulted in incomplete replica coverage after node failures. The system was later extended to explicitly backfill newly required replica ranges, ensuring that replication guarantees were restored after membership changes.

Gossip-based membership also presented challenges in failure recovery. A strict preference for contacting only *ALIVE* nodes prevented partially isolated nodes from repairing their membership view. The final design allowed fallback communication with other reachable peers, improving the system's ability to heal after network partitions or temporary disconnections.

Finally, ensuring correctness during node rejoin required careful handling of stale data. Rejoining nodes could incorrectly appear healthy before their data was fully restored. This was addressed by implementing post-rejoin backfill, ensuring that nodes recover their expected replica state before serving as fully consistent participants in the system.

These challenges highlight that distributed systems correctness is not determined solely by high-level design, but by careful handling of edge cases and interactions between components. The experience reinforced the importance of systematic validation, deterministic behavior, and strong invariants in building reliable distributed systems.

5 Related Work

Distributed storage systems have evolved through multiple architectural phases in response to increasing demands for scalability,

availability, and fault tolerance. Early systems were largely centralized, with data managed by a single node, as seen in traditional relational database systems [7]. While these systems provided strong consistency and well-defined data models, they were constrained by vertical scaling and suffered from a single point of failure.

To overcome these limitations, storage architectures transitioned to leader-based distributed systems, where a designated coordinator managed writes and maintained consistency across replicas. Systems such as ZooKeeper [8] and Bigtable [3] exemplify this approach, enabling replication and improved fault tolerance. However, these designs introduced new challenges, including write bottlenecks at the leader and reduced availability during leader election under network partitions.

Further evolution led to decentralized peer-to-peer architectures based on distributed hash tables, such as Chord [14] and Pastry [12]. These systems eliminated centralized coordination by distributing both data and responsibility across nodes, achieving high scalability and resilience to node churn. Consistent hashing techniques [9] enabled efficient data placement and minimized rebalancing overhead. However, early DHT-based designs often provided weak consistency guarantees and lacked controlled replication strategies for strong durability.

Distributed storage systems have since explored different trade-offs between consistency, availability, and scalability across these architectural paradigms. Systems such as Bigtable [3] emphasize structured storage and stronger consistency through centralized coordination, which can limit availability under failures.

The theoretical foundation for these trade-offs is captured by the CAP principle [2]. Earlier work on weakly consistent systems, such as Bayou [15], and studies on optimistic replication [13], provide mechanisms for handling conflicting updates and ensuring eventual convergence.

Scalable partitioning is commonly achieved using consistent hashing [9], which maps both data items and nodes into a shared hash space, enabling uniform load distribution and minimizing data movement when nodes join or leave the system. This approach is particularly effective in dynamic environments where membership changes are frequent.

Decentralized membership and failure detection are typically handled using gossip-based protocols [4, 6, 16]. Coordination across replicas in the absence of a global clock relies on lightweight ordering mechanisms and quorum-based replication. Logical time [11] provides a foundation for reasoning about event ordering without synchronized clocks, although many practical systems adopt simplified approaches such as timestamp-based ordering. Quorum-based replication [1] requires overlapping subsets of replicas to participate in read and write operations, enabling tunable consistency guarantees and allowing systems to balance latency, availability, and consistency under varying workloads and failure conditions.

This project builds on these foundations by combining decentralized partitioning, replication, quorum-based coordination, and gossip-based membership in a simplified system, enabling a focused evaluation of availability and consistency trade-offs.

6 Work Distribution

Development was divided by implementation ownership at the subsystem level so that each major path in the system had a clear primary maintainer. Rather than splitting work by broad features alone, responsibilities were assigned around the concrete mechanisms that determine placement, persistence, coordination, and recovery. The following breakdown reflects the primary engineering ownership for each major subsystem.

Yash was responsible for the consistent-hashing and placement path end to end. This included construction of the hash ring, virtual-node token generation, replica routing, and the actual mapping from an incoming client key to its ordered replica set on the ring. This work included the ring and vnode logic in `HashRingService` and `VirtualNodeService`, the request-to-replica lookup path in `ReplicaRoutingService`, and the ownership-based rebalance logic for node joins and graceful leaves. Yash also implemented the range-oriented rebalance support used when ownership changes, and added the internal HTTP endpoints used to expose cluster summaries and rebalance timing metrics for evaluation and presentation.

Smeet was responsible for the data plane and local persistence path. This included the local storage service, write-ahead log integration, in-memory indexing, and the logic for applying local and replica writes correctly under version ordering. In implementation terms, this work centered on the persistence and storage components such as `LocalStorageService`, `PersistentStorage`, `InMemoryStorageIndex`, and the replica-facing data application path. Smeet also built the main client-router-facing interaction path for replica GET and PUT operations and led the latency evaluation for read and write performance under varying quorum configurations. He was also responsible for the client application layer. This included implementing the client-router service for request routing, node registration, membership refresh, and client-side administrative operations.

Tarun was responsible for the control-plane membership and coordination mechanisms. This included gossip-based membership dissemination, failure detection, quorum coordination, and cluster reconfiguration behavior. The corresponding implementation centered on components such as `MembershipService`, `GossipService`, `FailureDetectorService`, and `QuorumCoordinatorService`, together with the logic that integrates membership changes into routing and recovery behavior. Tarun also designed and executed the evaluation methodology for node-failure handling, fault tolerance, and quorum-versus-latency trade-offs.

Although these ownership boundaries were clear, the final system required substantial cross-component integration. All three members participated in debugging distributed interactions across the storage plane, control plane, and routing path, especially in validating quorum behavior, replica reconstruction, and membership-triggered rebalancing under realistic cluster events.

7 Conclusion

This paper presented the design and implementation of a decentralized distributed key-value store that prioritizes availability and scalability under failures and dynamic cluster membership. By combining deterministic data partitioning, replication, quorum-based

coordination, and gossip-based membership, the system demonstrates how decentralized mechanisms can be used to maintain continuous operation without relying on centralized control.

The project also highlights the practical implications of these design choices. Quorum configurations directly influence system behavior, affecting both latency and availability, while decentralized membership and data redistribution enable the system to adapt efficiently to changes in cluster state. These observations emphasize the importance of carefully balancing consistency guarantees with performance requirements in distributed storage systems.

Overall, this work provides a focused exploration of the fundamental mechanisms underlying decentralized storage architectures. By isolating and implementing these core components in a simplified setting, the project offers insight into how availability-first systems operate and the trade-offs they introduce. This serves as a foundation for further exploration of more advanced consistency models, optimization techniques, and large-scale deployment scenarios.

8 Limitations

OrionKV uses timestamp-based last-write-wins for conflict resolution, which may result in lost updates under concurrent writes. The system does not provide strong consistency or support multi-key transactions, limiting its applicability for transactional workloads.

Rebalancing during node joins introduces overhead due to data transfer, and gossip-based membership provides eventual rather than instantaneous convergence, leading to temporary routing inconsistencies.

Finally, the system has been evaluated in a controlled environment with a limited number of nodes, and larger-scale deployments may expose additional performance and reliability challenges.

9 Future Work

Future work includes incorporating vector clocks for improved conflict resolution and adding hinted handoff to handle temporary replica unavailability. Anti-entropy mechanisms such as Merkle trees can further improve replica convergence.

Additional improvements include optimizing rebalancing, supporting larger-scale deployments, and exploring adaptive quorum configurations for different workload patterns.

Acknowledgments

The authors would like to thank Dr. Helen Gu, professor of CSC 724: Advanced Distributed Systems at North Carolina State University, for providing the opportunity and guidance to develop this project. We also express our sincere appreciation to the teaching assistant, Tural Mehtiyev, for his valuable support and efforts throughout the course.

References

- [1] Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica. 2012. Probabilistically bounded staleness for practical partial quorums. *VLDB* (2012).
- [2] Eric A. Brewer. 2000. Towards robust distributed systems. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*.
- [3] Fay Chang et al. 2006. Bigtable: A Distributed Storage System for Structured Data. In *OSDI*.

- [4] Abhinandan Das et al. 2002. SWIM: Scalable Weakly-Consistent Infection-Style Membership. In *DSN*.
- [5] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, et al. 2007. Dynamo: Amazon's Highly Available Key-Value Store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles*.
- [6] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, and John Larson. 1987. Epidemic algorithms for replicated database maintenance. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*.
- [7] Jim Gray and Andreas Reuter. 1992. Transaction Processing: Concepts and Techniques. (1992).
- [8] Patrick Hunt, Mahadev Konar, Flavio Junqueira, and Benjamin Reed. 2010. ZooKeeper: Wait-free Coordination for Internet-scale Systems. In *USENIX ATC*.
- [9] David Karger et al. 1997. Consistent Hashing and Random Trees. In *PODC*.
- [10] Avinash Lakshman and Prashant Malik. 2009. Cassandra: A Decentralized Structured Storage System. In *Proceedings of the 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems*.
- [11] Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. *Commun. ACM* (1978).
- [12] Antony Rowstron and Peter Druschel. 2001. Pastry: Scalable, Distributed Object Location and Routing for Large-Scale Peer-to-Peer Systems. In *Middleware*.
- [13] Yasushi Saito and Marc Shapiro. 2005. Optimistic Replication. *Comput. Surveys* (2005).
- [14] Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. 2001. Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications. In *Proceedings of ACM SIGCOMM*.
- [15] Douglas Terry et al. 1995. Managing Update Conflicts in Bayou. In *SOSP*.
- [16] Robbert van Renesse et al. 2003. Gossip-Style Failure Detection. In *Middleware*.
- [17] Sai Tarun Yellamraju, Smeet Nagda, and Yash Mor. 2026. OrionKV: A Decentralized Distributed Key-Value Store. <https://github.com/SaiTarun314/OrionKV>. GitHub repository.